



Student Management System

A Beginner-Friendly Python Mini Project (OOP + Functions)

This guide walks students through building a menu-driven **Student Management System** in Python using **two classes**: a **Student** class that represents one student as an object, and a **StudentManagementSystem** class that manages a collection of these objects (add, view, update, delete, search). The project covers core programming concepts such as classes, objects, dictionaries, loops, conditionals, and functions — making it an ideal beginner/intermediate practice project or classroom assignment.

Project Objectives

- Understand how to design one class to represent data (Student) and another class to manage that data (StudentManagementSystem).
- Create a separate **Student object** for every student, instead of storing each record as a plain dictionary.
- Implement CRUD operations: Add, View, Update, Delete and Search records.
- Build a simple, menu-driven command-line interface (CLI) using loops and conditionals.
- Learn good coding practices such as input validation and clear function naming.

Prerequisites

- Python 3.x installed on the computer (download from python.org).
- A code editor such as VS Code, PyCharm, or IDLE.
- Basic understanding of Python syntax: variables, if-else, and loops.

Tip for the classroom: Ask students to add and test one method at a time (Steps 4-8 below) via the menu, rather than writing the entire class in one go.



Step-by-Step Instructions

This is a pure Python, console-based (command-line) project — no GUI or external library is used. Every student is stored as an **object of the Student class** (not a plain dictionary), and the **StudentManagementSystem** class keeps all these objects in one dictionary. Every step below shows exactly what code to add and explains what it does.

Step 1: Create the Project File

Create a new file named **student_management.py**. The entire program — two classes, several functions — will live in this single file. No pip installs or extra libraries are required; it uses only core Python.

Step 2: Define the Student Class

First, create a small class called **Student**. Its only job is to represent **one student** as an object — every time a new student is added, a new **Student** object will be created to hold that student's details. Add a **display()** method too, so any Student object can print itself.

```
class Student:
    """Represents a single student as an object."""

    def __init__(self, roll_no, name, age, grade):
        self.roll_no = roll_no
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print(f"Roll No: {self.roll_no} | Name: {self.name} | "
              f"Age: {self.age} | Grade: {self.grade}")
```

- Each Student **object** stores its own roll_no, name, age and grade as attributes (**self.name**, **self.age**, etc.).
- **display()** is a method on the object itself, so printing a student's details later is as simple as calling **student.display()**.

Step 3: Define the StudentManagementSystem Class and Constructor

Now create the second class, **StudentManagementSystem**, which manages many **Student** objects. Its **__init__** constructor sets up one dictionary, **self.students**, that will map each roll number to its own **Student** object.

```
class StudentManagementSystem:
    def __init__(self):
        # Dictionary to store Student objects: {roll_no: Student object}
        self.students = {}
```

- Unlike storing plain dictionaries, **self.students** now holds a **Student object** for every roll number.
- This keeps the data (Student) and the operations on that data (StudentManagementSystem) cleanly separated.



Step 4: Write the `add_student()` Function

This function asks the user for the roll number, name, age and grade, checks that the roll number isn't already used, and then creates a brand-new **Student object** to represent that student.

```
def add_student(self):
    roll_no = input("Enter Roll Number: ").strip()
    if roll_no in self.students:
        print("A student with this roll number already exists!")
        return
    name = input("Enter Name: ").strip()
    age = input("Enter Age: ").strip()
    grade = input("Enter Grade/Class: ").strip()
    self.students[roll_no] = Student(roll_no, name, age, grade)
    print("Student added successfully!")
```

- **`.strip()`** removes accidental leading/trailing spaces from typed input.
- The **`if roll_no in self.students`** check prevents two students from accidentally being saved under the same roll number.
- **`Student(roll_no, name, age, grade)`** creates a new object, and it is stored under its roll number in **`self.students`**.

Step 5: Write the `view_students()` Function

This function prints every student currently stored, one line per student, by calling each Student object's own **`display()`** method.

```
def view_students(self):
    if not self.students:
        print("No student records found.")
        return
    print("\n--- Student Records ---")
    for student in self.students.values():
        student.display()
```

- The empty-dictionary check prints a friendly message instead of nothing when no students have been added yet.
- **`self.students.values()`** gives every Student object directly (the roll numbers aren't needed here since each object already stores its own).



Step 6: Write the `update_student()` Function

This function looks up a student by roll number to get their **Student object**, then lets the user retype any field. If the user just presses Enter (leaves it blank), the old value is kept unchanged.

```
def update_student(self):
    roll_no = input("Enter Roll Number to update: ").strip()
    if roll_no not in self.students:
        print("Student not found!")
        return
    print("Leave a field blank to keep its current value.")
    student = self.students[roll_no]
    name = input(f"New Name [{student.name}]: ").strip()
    age = input(f"New Age [{student.age}]: ").strip()
    grade = input(f"New Grade [{student.grade}]: ").strip()
    if name:
        student.name = name
    if age:
        student.age = age
    if grade:
        student.grade = grade
    print("Student record updated successfully!")
```

- **student = self.students[roll_no]** gets the actual Student object, so the code below can change its attributes directly.
- Each **if name:** check is True only when the user actually typed something, so a blank answer leaves that attribute untouched.

Step 7: Write the `delete_student()` Function

This function removes a student's object completely from the dictionary, using Python's built-in **del** keyword, after confirming the roll number exists.

```
def delete_student(self):
    roll_no = input("Enter Roll Number to delete: ").strip()
    if roll_no in self.students:
        del self.students[roll_no]
        print("Student deleted successfully!")
    else:
        print("Student not found!")
```

Step 8: Write the `search_student()` Function

This function looks up one student's object by roll number and calls its **display()** method, without showing the whole list.

```
def search_student(self):
    roll_no = input("Enter Roll Number to search: ").strip()
    if roll_no in self.students:
        self.students[roll_no].display()
    else:
        print("Student not found!")
```



Step 9: Write the menu() Function

This is the driver function that ties everything together. It shows a numbered list of options inside a **while True** loop, reads the user's choice, and calls the matching function above. Choosing option 6 breaks out of the loop and ends the program.

```
def menu(self):
    while True:
        print("\n==== Student Management System =====")
        print("1. Add Student")
        print("2. View All Students")
        print("3. Update Student")
        print("4. Delete Student")
        print("5. Search Student")
        print("6. Exit")

        choice = input("Enter your choice (1-6): ").strip()

        if choice == "1":
            self.add_student()
        elif choice == "2":
            self.view_students()
        elif choice == "3":
            self.update_student()
        elif choice == "4":
            self.delete_student()
        elif choice == "5":
            self.search_student()
        elif choice == "6":
            print("Exiting the system. Goodbye!")
            break
        else:
            print("Invalid choice! Please enter a number between 1 and 6.")
```

- **while True:** keeps showing the menu again after every action, until the user chooses to exit.
- **break** is the only way out of the loop, which happens only when the user enters 6.

Step 10: Add the Program Entry Point

At the very bottom of the file (outside both classes, with no indentation), add this. It creates one **StudentManagementSystem** object and calls its **menu()** method — this is what actually starts the program when the file is run.

```
system = StudentManagementSystem()
system.menu()
```

- **system** is an object (instance) of the **StudentManagementSystem** class, created by calling the class like a function.
- **system.menu()** calls the menu method on that object, which starts the loop shown in Step 9.

Step 11: Run and Test the Program

Save the file, open a terminal in the same folder, and run it with the command below. Then try every option — add, view, update, delete, and search — with a few sample records to confirm everything works.

```
python student_management.py
```



Complete Source Code

Below is the complete, ready-to-run code for **student_management.py**, combining all the steps above into the **Student** and **StudentManagementSystem** classes.

```
class Student:
    """Represents a single student as an object."""

    def __init__(self, roll_no, name, age, grade):
        self.roll_no = roll_no
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print(f"Roll No: {self.roll_no} | Name: {self.name} | "
              f"Age: {self.age} | Grade: {self.grade}")

class StudentManagementSystem:
    """A simple menu-driven Student Management System."""

    def __init__(self):
        # Dictionary to store Student objects: {roll_no: Student object}
        self.students = {}

    def add_student(self):
        roll_no = input("Enter Roll Number: ").strip()
        if roll_no in self.students:
            print("A student with this roll number already exists!")
            return
        name = input("Enter Name: ").strip()
        age = input("Enter Age: ").strip()
        grade = input("Enter Grade/Class: ").strip()
        self.students[roll_no] = Student(roll_no, name, age, grade)
        print("Student added successfully!")

    def view_students(self):
        if not self.students:
            print("No student records found.")
            return
        print("\n--- Student Records ---")
        for student in self.students.values():
            student.display()

    def update_student(self):
        roll_no = input("Enter Roll Number to update: ").strip()
        if roll_no not in self.students:
            print("Student not found!")
            return
        print("Leave a field blank to keep its current value.")
        student = self.students[roll_no]
        name = input(f"New Name [{student.name}]: ").strip()
        age = input(f"New Age [{student.age}]: ").strip()
        grade = input(f"New Grade [{student.grade}]: ").strip()
        if name:
            student.name = name
        if age:
            student.age = age
        if grade:
            student.grade = grade
```



```
print("Student record updated successfully!")

def delete_student(self):
    roll_no = input("Enter Roll Number to delete: ").strip()
    if roll_no in self.students:
        del self.students[roll_no]
        print("Student deleted successfully!")
    else:
        print("Student not found!")

def search_student(self):
    roll_no = input("Enter Roll Number to search: ").strip()
    if roll_no in self.students:
        self.students[roll_no].display()
    else:
        print("Student not found!")

def menu(self):
    while True:
        print("\n===== Student Management System =====")
        print("1. Add Student")
        print("2. View All Students")
        print("3. Update Student")
        print("4. Delete Student")
        print("5. Search Student")
        print("6. Exit")

        choice = input("Enter your choice (1-6): ").strip()

        if choice == "1":
            self.add_student()
        elif choice == "2":
            self.view_students()
        elif choice == "3":
            self.update_student()
        elif choice == "4":
            self.delete_student()
        elif choice == "5":
            self.search_student()
        elif choice == "6":
            print("Exiting the system. Goodbye!")
            break
        else:
            print("Invalid choice! Please enter a number between 1 and 6.")

system = StudentManagementSystem()
system.menu()
```



Sample Program Output

A typical session with the program looks like this (user input shown after the prompts):

```
===== Student Management System =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter your choice (1-6): 1
Enter Roll Number: 101
Enter Name: Aarav Sharma
Enter Age: 15
Enter Grade/Class: 10th
Student added successfully!

===== Student Management System =====
Enter your choice (1-6): 2

--- Student Records ---
Roll No: 101 | Name: Aarav Sharma | Age: 15 | Grade: 10th

===== Student Management System =====
Enter your choice (1-6): 6
Exiting the system. Goodbye!
```

Suggested Assessment Checklist

Criteria	Marks
Student class defined correctly (attributes + display())	10
StudentManagementSystem class and __init__ correctly defined	10
Add Student function works correctly	15
View Students function works correctly	10
Update Student function works correctly	15
Delete Student function works correctly	10
Search Student function works correctly	10
Menu loop and program flow	10
Code readability & comments	10
Total	100